Viktoria Ivanova - 7MI3400799

Selling prices of villas in Landvetter

In this task, a linear regression model is build to check the relation between living areas and selling prices in Landvetter. Data will be processed in the best way for the needs of the tasks, in order to retrieve the most valuable information.

In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
```

Loading the data:

In [3]:
```
living_prices_df = pd.read_csv("data_assignment2.csv")
living_prices_df
```

Out[3]:

	ID	Living_area	Rooms	Land_size	Biarea	Age	Selling_price
0	1	104	5.0	271.0	25.0	33	4600000
1	2	99	5.0	1506.0	6.0	88	4450000
2	3	133	6.0	486.0	NaN	44	4900000
3	4	175	7.0	728.0	NaN	14	6625000
4	5	118	6.0	1506.0	NaN	29	4600000
5	6	133	6.0	823.0	NaN	12	6650000
6	7	70	4.0	1685.0	70.0	53	2800000
7	8	134	6.0	1593.0	49.0	57	6100000
8	9	70	5.0	1120.0	102.0	81	3000000
9	10	133	6.0	1503.0	NaN	51	3000000
10	11	121	4.0	1575.0	112.0	81	4000000
11	12	136	6.0	381.0	NaN	42	4350000
12	13	86	4.0	1529.0	30.0	90	3025000
13	14	135	6.0	334.0	8.0	45	5215000
14	15	130	5.0	2095.0	NaN	63	4275000
15	16	104	5.0	399.0	13.0	59	4100000
16	17	66	2.0	1655.0	20.0	90	3100000
17	18	129	5.0	414.0	21.0	32	5370000
18	19	75	NaN	1698.0	NaN	75	3000000
19	20	160	6.0	2104.0	NaN	63	4175000
20	21	87	4.0	268.0	NaN	22	3850000
21	22	182	7.0	1851.0	50.0	4	6300000
22	23	135	5.0	224.0	7.0	3	5350000
23	24	100	4.0	1213.0	84.0	65	4370000
24	25	202	6.0	1253.0	NaN	54	4250000
25	26	136	5.0	1381.0	8.0	21	4800000
26	27	185	7.0	746.0	31.0	44	5940000
27	28	153	6.0	3285.0	43.0	96	6600000
28	29	135	5.0	223.0	NaN	2	5200000
29	30	73	4.0	1944.0	70.0	53	3925000
30	31	120	4.0	1679.0	10.0	51	4350000
31	32	122	5.0	567.0	NaN	7	5825000
32	33	63	NaN	1665.0	60.0	39	2500000

	ID	Living_area	Rooms	Land_size	Biarea	Age	Selling_price
33	34	150	5.0	1335.0	NaN	8	6050000
34	35	210	6.0	1978.0	NaN	5	6100000
35	36	108	5.0	299.0	NaN	32	4800000
36	37	111	4.0	367.0	NaN	59	4200000
37	38	72	4.0	1645.0	50.0	62	3450000
38	39	170	7.0	626.0	30.0	40	6195000
39	40	142	6.0	700.0	22.0	32	6050000
40	41	170	7.0	NaN	NaN	0	1775000
41	42	120	4.0	891.0	NaN	6	5850000
42	43	107	4.0	1643.0	NaN	59	4960000
43	44	135	5.0	223.0	8.0	2	5200000
44	45	145	5.0	1363.0	NaN	13	6150000
45	46	168	6.0	656.0	7.0	81	2360000
46	47	69	3.0	1500.0	69.0	82	2700000
47	48	170	7.0	1098.0	20.0	51	5930000
48	49	160	6.0	1191.0	NaN	2	6800000
49	50	170	7.0	1637.0	25.0	11	4850000
50	51	158	6.0	1539.0	23.0	12	5000000
51	52	103	5.0	264.0	NaN	19	4245000
52	53	148	5.0	771.0	30.0	32	5725000
53	54	139	5.0	218.0	NaN	3	5275000
54	55	118	6.0	937.0	118.0	45	4850000
55	56	159	7.0	1315.0	30.0	64	4825000

We can see relatively small amount of data, it cannot be sufficient for totally clear predictions and conclusions but a model can be created, based on the records.

In [4]:
```
living_prices_df.describe().T
```

Out[4]:

	count	mean	std	min	25%	50%	
ID	56.0	2.850000e+01	1.630951e+01	1.0	14.75	28.5	
Living_area	56.0	1.286786e+02	3.600662e+01	63.0	104.00	133.0	
Rooms	54.0	5.296296e+00	1.126518e+00	2.0	5.00	5.0	
Land_size	55.0	1.125455e+03	6.566852e+02	218.0	526.50	1213.0	
Biarea	32.0	3.909375e+01	3.167003e+01	6.0	18.25	30.0	
Age	56.0	4.076786e+01	2.807910e+01	0.0	12.75	43.0	
Selling_price	56.0	4.713125e+06	1.241117e+06	1775000.0	4075000.00	4812500.0	583

Let's see the ammount of missing data:

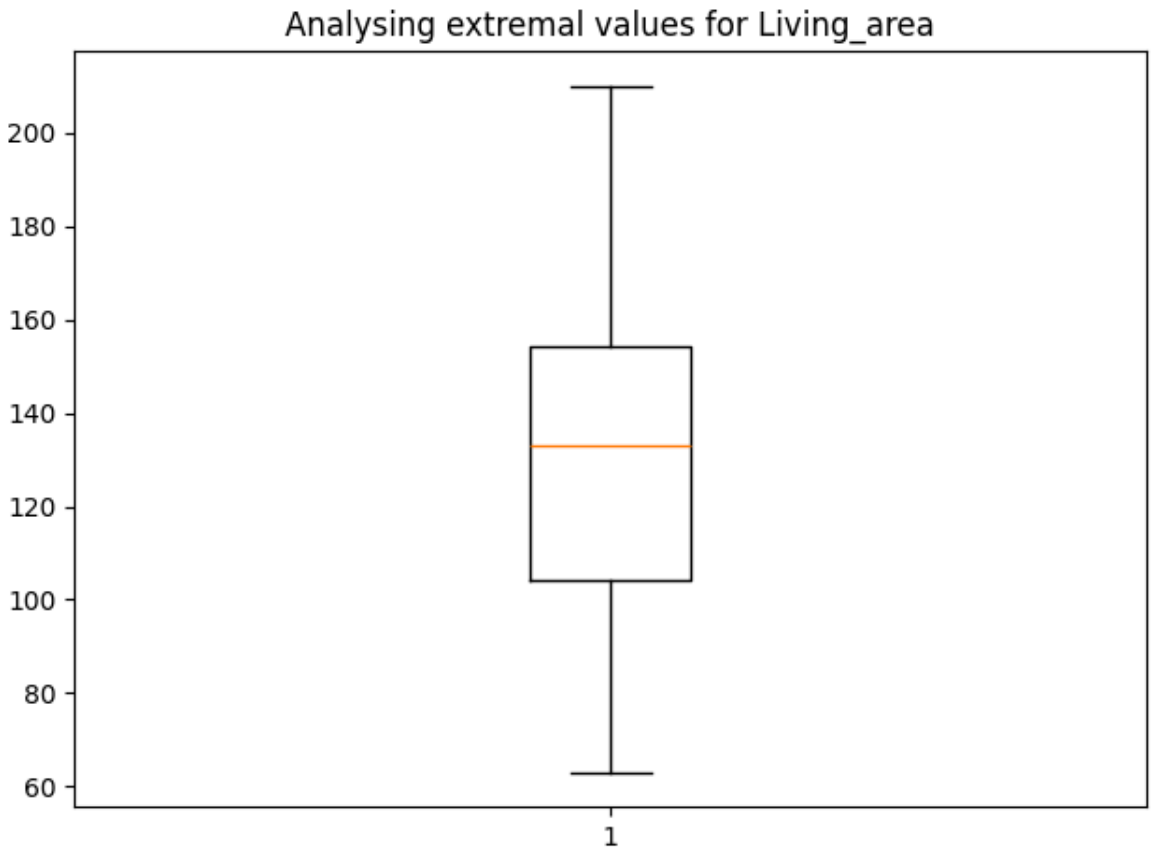
In [5]:
```
missing_values = living_prices_df.isna().mean().sort_values(ascending=False)
missing_values
```

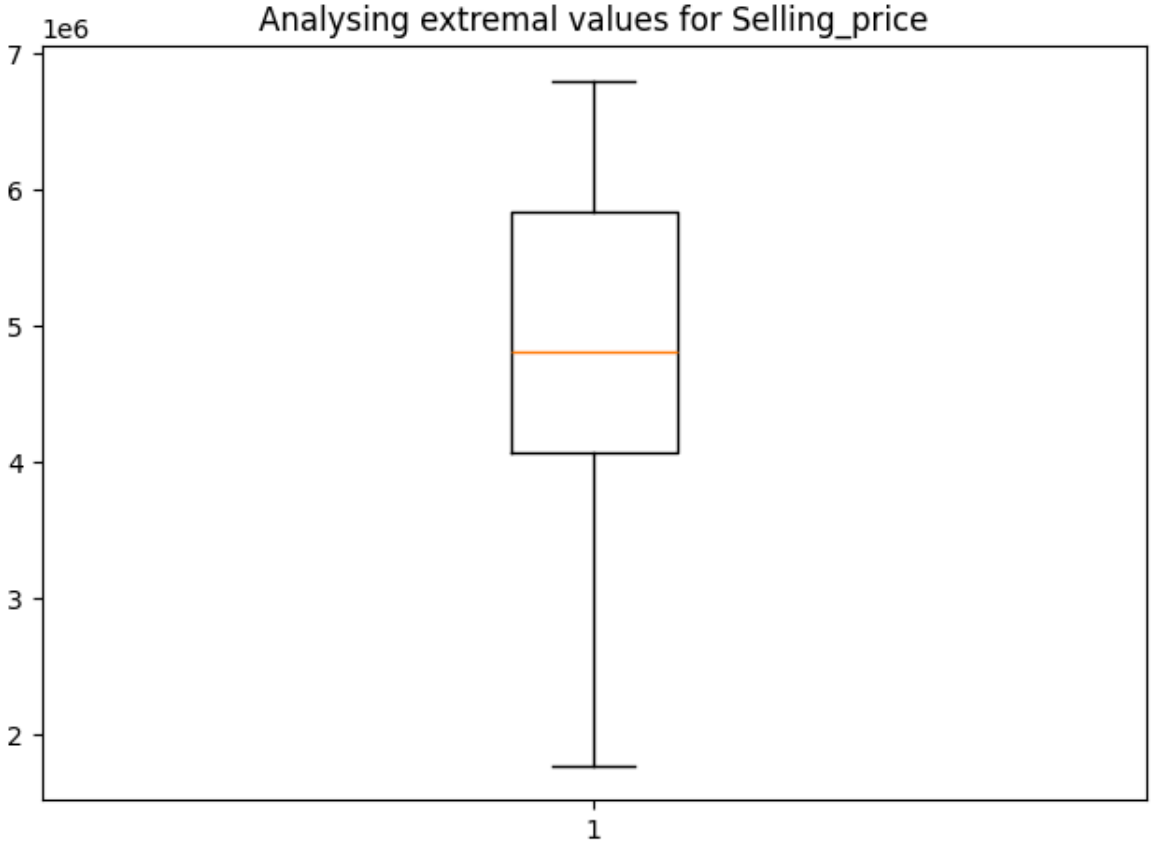
Out[5]:
```
Biarea           0.428571
Rooms            0.035714
Land_size        0.017857
ID               0.000000
Living_area      0.000000
Age              0.000000
Selling_price    0.000000
dtype: float64
```

Based on the result, we are going to check the dependency between the Living are and Selling price features, so we don't need to drop or fill the rows with their mean values.

Is is important to check for extremal values which can alter significantly the results and wrong assumptions might be made.

In [6]:
```
extremal_values_check = living_prices_df[['Living_area', 'Selling_price']]

for label, data in extremal_values_check.items():
    plt.title(f'Analysing extremal values for {label}')
    plt.boxplot(data)
    plt.tight_layout()
    plt.show()
```





In [7]:
```
living_prices_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 56 entries, 0 to 55
Data columns (total 7 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   ID             56 non-null     int64
 1   Living_area    56 non-null     int64
 2   Rooms          54 non-null     float64
 3   Land_size      55 non-null     float64
 4   Biarea         32 non-null     float64
 5   Age            56 non-null     int64
 6   Selling_price  56 non-null     int64
dtypes: float64(3), int64(4)
memory usage: 3.2 KB
```

Let's create a Linear regression model.

In [8]:
```
lin_regression = LinearRegression()
X_living_area = living_prices_df[['Living_area']]
y_selling_prices = living_prices_df[['Selling_price']]
lin_regression.fit(X_living_area, y_selling_prices)
predictions = lin_regression.predict(X_living_area)

plt.figure(figsize=(8, 6))
plt.scatter(X_living_area, y_selling_prices)
plt.title('Relationship between living area and selling price in Landvetter')
plt.plot(X_living_area, predictions, 'r')
plt.xlabel('Living area')
plt.ylabel('Selling price ($)')
plt.tight_layout()
plt.show()
```

In [9]:
```
r2 = lin_regression.score(X_living_area, y_selling_prices)
r2
```

Out[9]:
```
0.31579478122912397
```

The R^2 score is pretty bad, however, with more data, better results can be received.

b. What are the values of the slope and intercept of the regression line?

In [15]:
```
print("Slope value: ", lin_regression.coef_[0])
print("Intercept: ", lin_regression.intercept_)
```

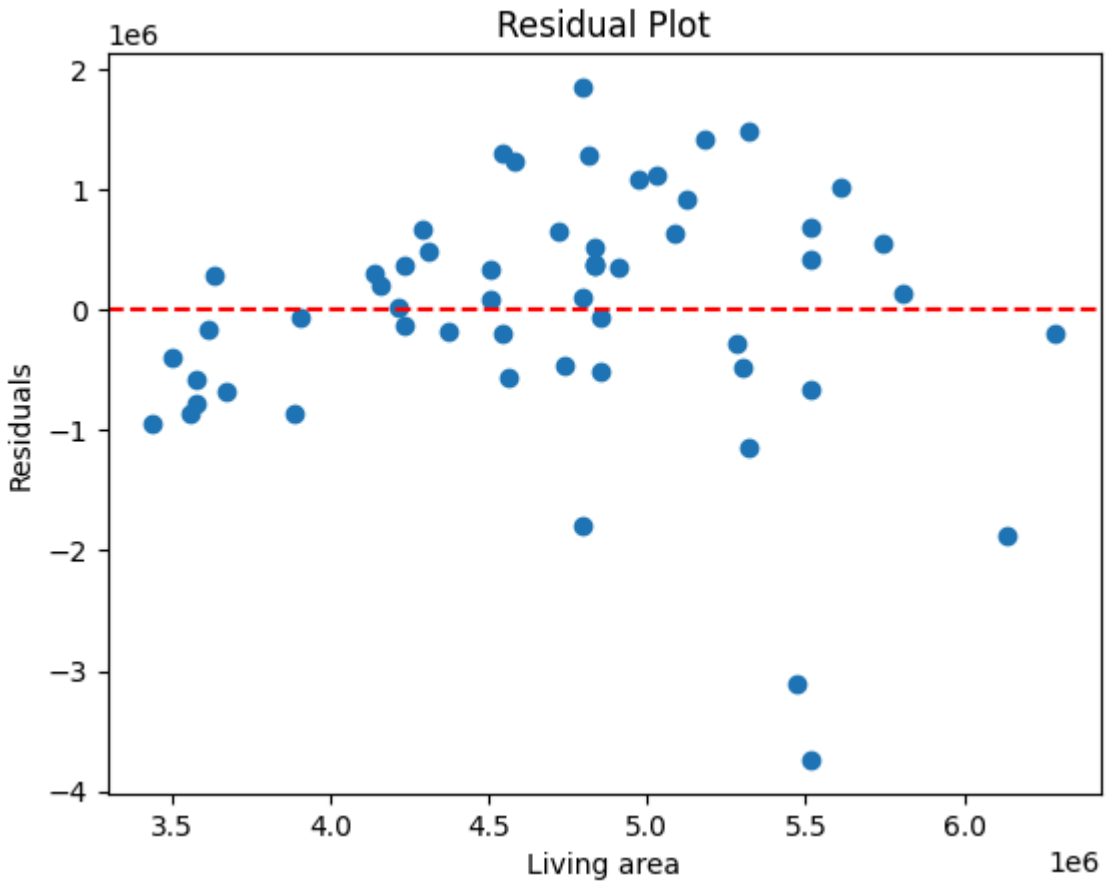
```
Slope value:  [19370.13854733]
Intercept:  [2220603.24335587]
```

c. Use this model to predict the selling prices of houses which have living area 100 m2, 150 m2 and 200 m2.

In [ ]:
```
areas = [100, 150, 200]

areas_df = pd.DataFrame(areas, columns=['Living_area'])

area_predictions = lin_regression.predict(areas_df)
print(area_predictions)
```

d. Draw a residual plot.

In [13]:
```
residuals = y_selling_prices - predictions


plt.scatter(predictions, residuals)
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel('Living area')
plt.ylabel('Residuals')
plt.title('Residual Plot')
plt.show()
```



A lot of spreads are visual in the residual plot, so we can conclude that there is a dependency between the living area and the selling prices in Landvetter, but more records should be collected to be able to create a good investigation with valuable results.

Iris dataset classification

In this project, we will investigate and process the iris dataset, included in sklearn.datasets, in order to be able to give answers to some ongoing questions about its classification and scores.

Imports

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report
from sklearn.metrics import accuracy_score
```

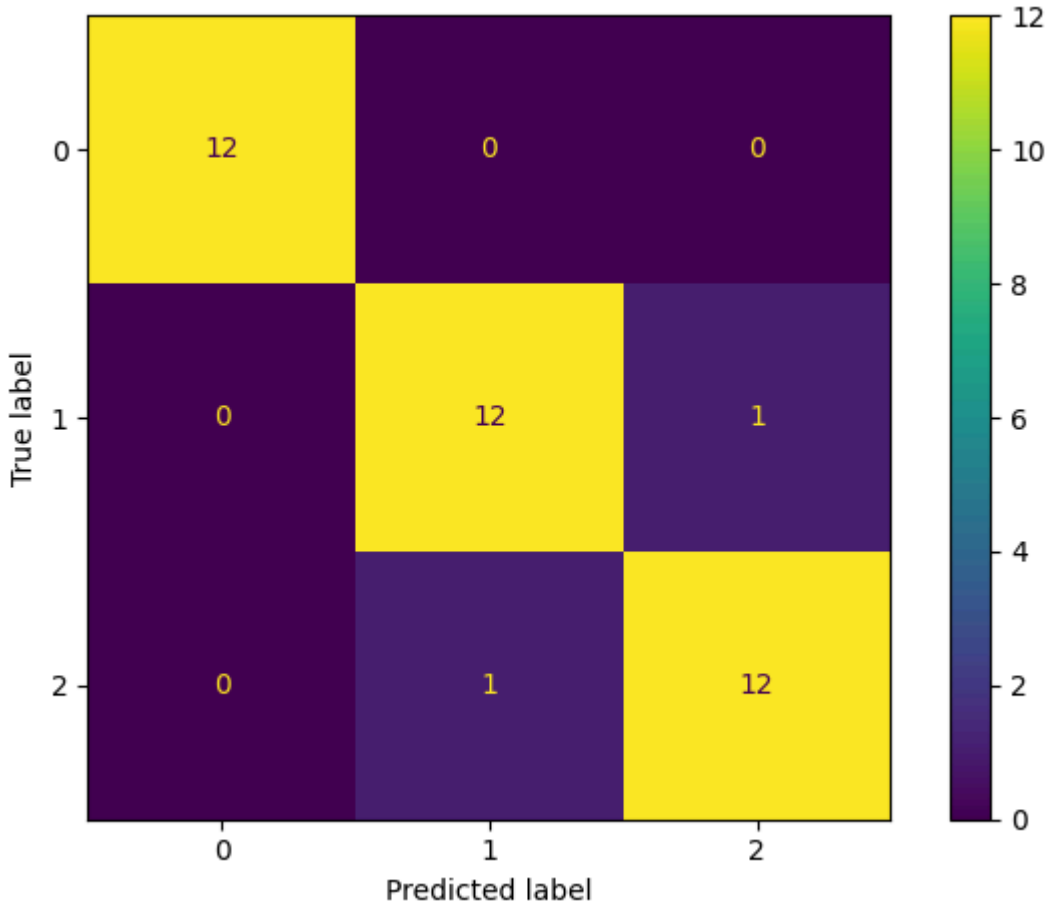
In [ ]:
```
iris_dataset = load_iris()

X = iris_dataset.data
y = iris_dataset.target
target_names = iris_dataset.target_names
target_names
```

```
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

a. Use a confusion matrix to evaluate the use of logistic regression to classify the iris data set

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random

logistic_regression = LogisticRegression(random_state=42, max_iter=300)
logistic_regression.fit(X_train , y_train)
y_pred = logistic_regression.predict(X_test)
print(accuracy_score(y_test, y_pred))

ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.tight_layout()
plt.show()
print(classification_report(y_test, y_pred, target_names=target_names))
```

```
0.9473684210526315
```



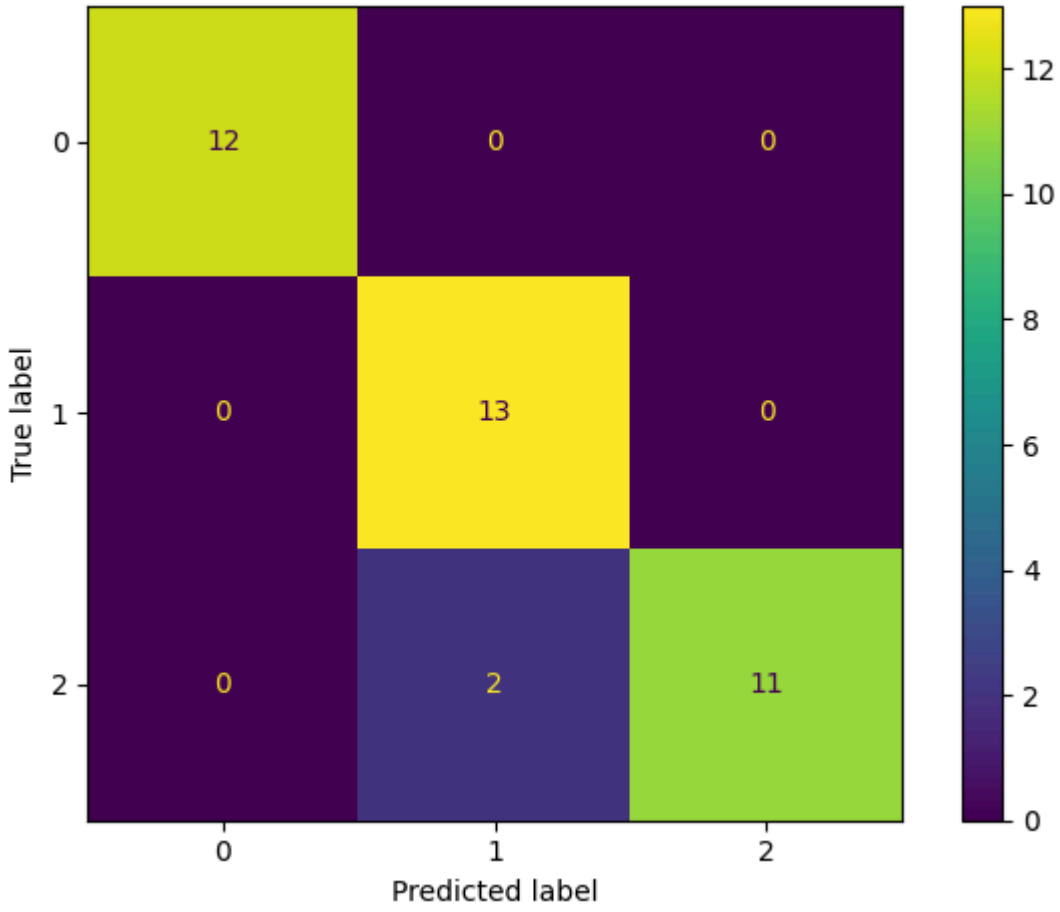
```
                precision    recall  f1-score   support

        setosa       1.00      1.00      1.00        12
    versicolor       0.92      0.92      0.92        13
     virginica       0.92      0.92      0.92        13

      accuracy                           0.95        38
     macro avg       0.95      0.95      0.95        38
  weighted avg       0.95      0.95      0.95        38
```

We can conclude that the setosa is predicted correctly, virginica has 3 missclassified and versicolor has 1 missclassified

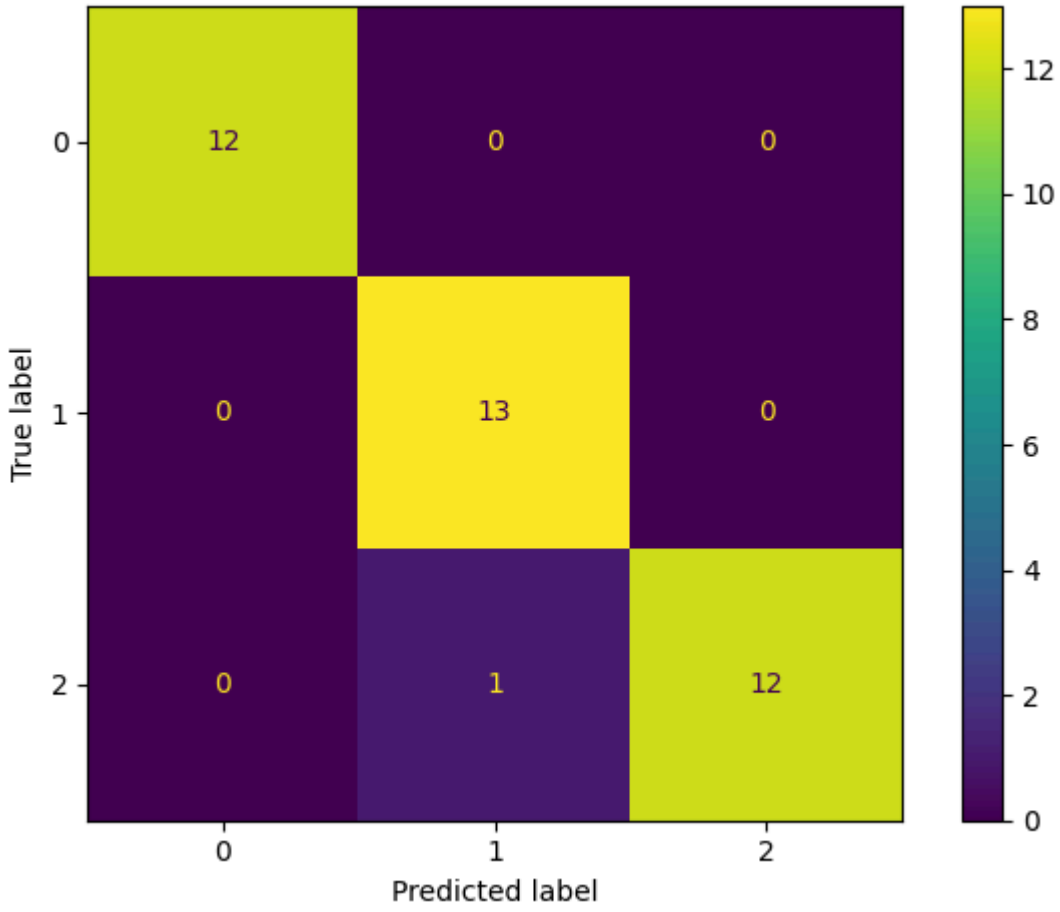
b. Use k-nearest neighbours to classify the iris data set with some different values for k, and with uniform and distance-based weights. What will happen when k grows larger for the different cases? Why?

In [ ]:
```
knn_values = [1, 3, 7, 9, 13, 21]

for k in knn_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)

    y_pred_knn = knn.predict(X_test)
    print("KNN for k =", k,)

    ConfusionMatrixDisplay.from_predictions(y_test, y_pred_knn)
    plt.tight_layout()
    plt.show()

    print(accuracy_score(y_test, y_pred_knn))
    print(classification_report(y_test, y_pred_knn, target_names=target_names))
```

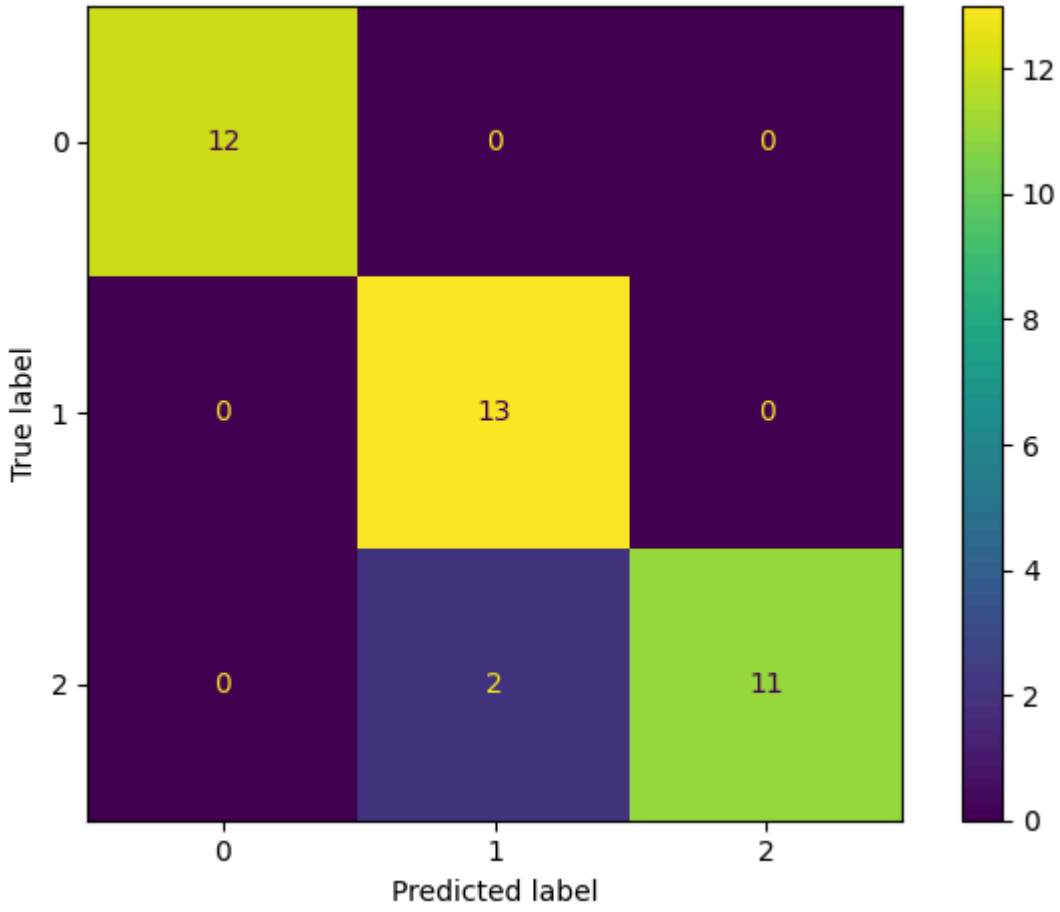
```
KNN for k = 1
```



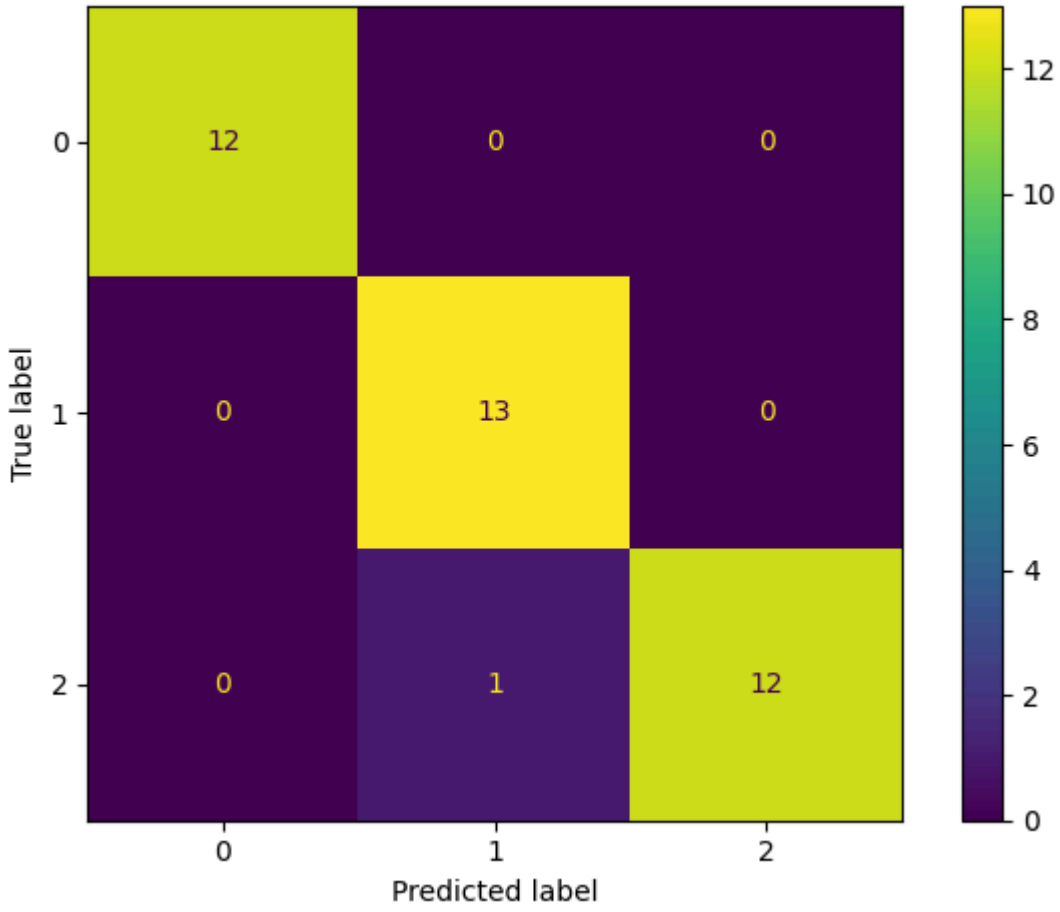

```
0.9473684210526315
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        12
  versicolor       0.87      1.00      0.93        13
   virginica       1.00      0.85      0.92        13

    accuracy                           0.95        38
   macro avg       0.96      0.95      0.95        38
weighted avg       0.95      0.95      0.95        38

KNN for k = 3
```



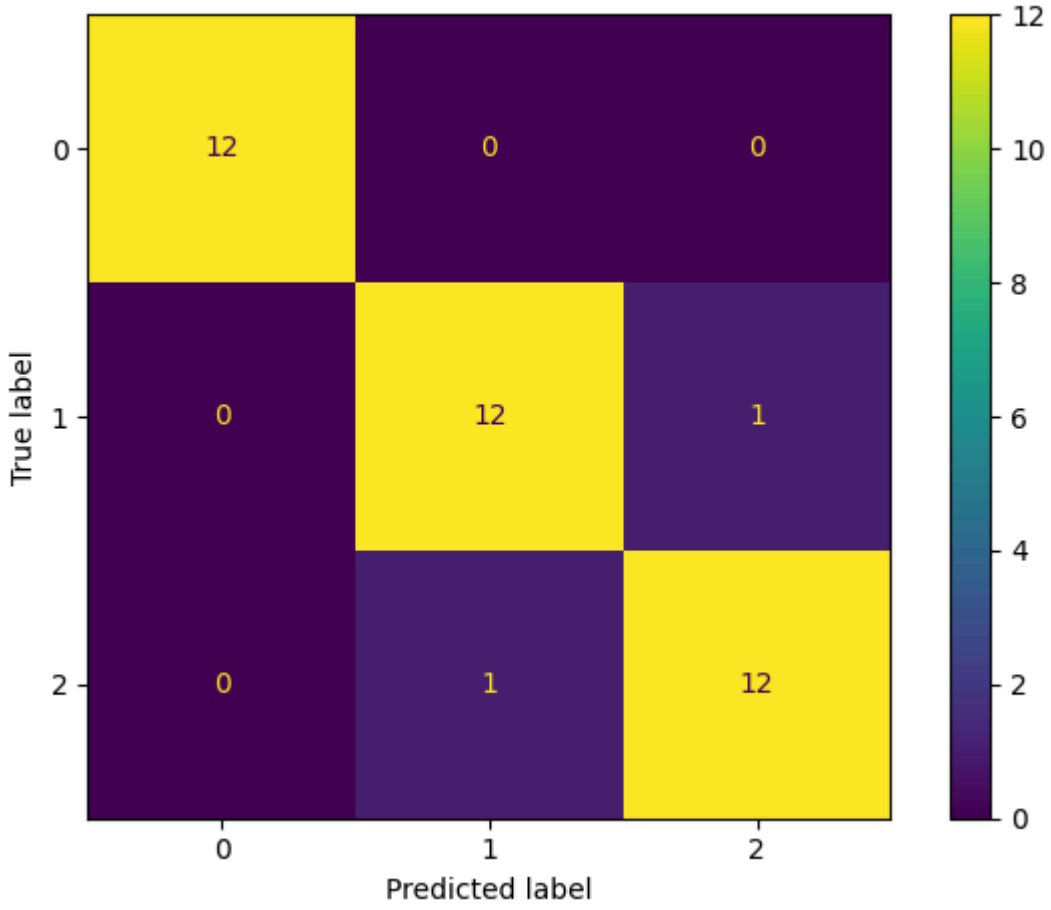

```
0.9736842105263158
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        12
  versicolor       0.93      1.00      0.96        13
   virginica       1.00      0.92      0.96        13

    accuracy                           0.97        38
   macro avg       0.98      0.97      0.97        38
weighted avg       0.98      0.97      0.97        38

KNN for k = 7
```



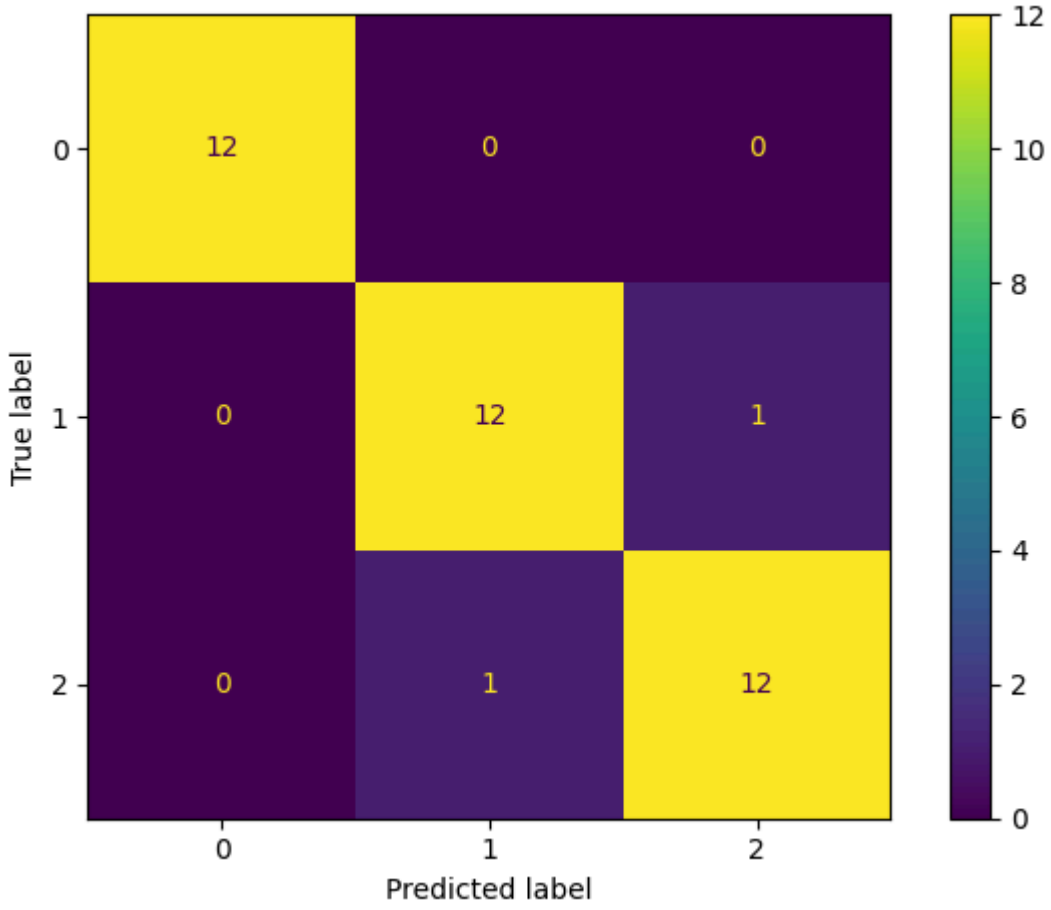

```
0.9473684210526315
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        12
  versicolor       0.87      1.00      0.93        13
   virginica       1.00      0.85      0.92        13

    accuracy                           0.95        38
   macro avg       0.96      0.95      0.95        38
weighted avg       0.95      0.95      0.95        38

KNN for k = 9
```




```
0.9736842105263158
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        12
  versicolor       0.93      1.00      0.96        13
   virginica       1.00      0.92      0.96        13

    accuracy                           0.97        38
   macro avg       0.98      0.97      0.97        38
weighted avg       0.98      0.97      0.97        38

KNN for k = 13
```




```
0.9473684210526315
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        12
  versicolor       0.92      0.92      0.92        13
   virginica       0.92      0.92      0.92        13

    accuracy                           0.95        38
   macro avg       0.95      0.95      0.95        38
weighted avg       0.95      0.95      0.95        38

KNN for k = 21
```




```
0.9473684210526315
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        12
  versicolor       0.92      0.92      0.92        13
   virginica       0.92      0.92      0.92        13

    accuracy                           0.95        38
   macro avg       0.95      0.95      0.95        38
weighted avg       0.95      0.95      0.95        38
```

The best result is for k = 9, this is based on the result score and the visible confusion matrices. Larger k values cause over smoothing and lower accuracy.

In conclusion, Both Logistic regression and KNN result in high scores which means that the models are predicting very well. The only problem with KNN is that for too large k, it causes loss of detail and worse performance.